Backend algorithms reference

Updated November 2, 2026 for schema `0005_protected_workflows`. This reference describes implemented decision procedures. Pseudocode explains the contracts; the linked Python modules remain authoritative. The companion formula reference defines scores and bounds, and the technical guide maps components.

1. Engagement extraction and planning

Sources: `engagement_parser.py`, `document_text.py`, `planner.py`, `exploit_planner.py` and `planners.py` under `backend/modules/`.

Document extraction is bounded and returns warnings when input is incomplete. Deterministic parsing extracts candidate assets, scope, exclusions, permissions and engagement rules. Operators review the parsed brief before registration. Exclusion text must not be interpreted as a target list.

Recon planning uses deterministic templates when no usable provider result is available. A normal host template has eight candidate steps; scope, capability and policy filtering can reduce that set. CIDR targets use discovery/service sweeps instead of treating the network as one application endpoint.

Every proposed step passes the same policy boundary before becoming executable. Verification and campaign plans depend on the analyzed evidence and current phase. No parser, model response or proposal constitutes execution approval.

```
extract bounded document text and tables
parse targets, allowed scopes, exclusions and restrictions
operator reviews and registers the engagement
draft bounded candidate steps for the current phase
validate each candidate against policy and capabilities
save a versioned plan; await explicit per-step approval
```

2. Command and scope validation

Sources: `policy_engine.py`, `command_values.py`, `scope_rules.py`.

Validation parses command arguments, matches a supported tool, checks all flag occurrences and their values, and extracts recognized destinations. Tool bans, capabilities, exploitation/lab gates, engagement windows and exclusions apply. A safe later flag cannot conceal an earlier unsafe value.

For network scopes, a requested network must be a subnet of an authorized network; validating only its first address is insufficient. Exact hostname scopes match exactly. An explicit `*.example.test` wildcard matches subdomains, not the apex. URL scopes check scheme, effective port and path boundaries and reject decoded dot traversal. The shared bare host:port helper does not itself provide universal port isolation; promotion separately rejects port-only scope.

Authorization must hold again when the worker starts. Command-line checks do not implement a general firewall against scanner internals or future DNS changes.

```
require supported tool, capabilities and permitted phase
validate every option and bounded value
collect recognized primary and secondary destinations
require allowed scope containment for each destination
refuse overlap with excluded scope
require active engagement window and applicable permissions
return permitted command or a specific refusal
```

3. Plan edits and dependency invalidation

Sources: `main.py`, `phases.py`, `workflow.py`.

A plan save checks the supplied plan version. Executed steps cannot change their position, tool, command, phase or enabled state. Idle unexecuted steps remain editable. The phase ordering depends on whether lab campaign mode is enabled.

Changes affecting execution identify the earliest affected phase; its findings, analysis and dependent state are invalidated while earlier findings survive. An unchanged save advances the plan version but preserves the current report. A reason-only edit preserves analysis and invalidates current reporting.

```
require expected plan_version and idle editable state
normalize candidate plan and validate executed-step invariants
if execution-bearing changes:
    invalidate earliest affected phase and dependents
else if only descriptive reasons changed:
    invalidate current report
increment plan version and commit under workflow revision
```

4. Durable execution, attempts and recovery

Sources: `jobs.py`, `executor.py`, `ssh_executor.py`, `worker.py`.

Approval creates or reuses the unique execution row for a plan step. Conditional claims coordinate queue consumers. Before execution, the worker resolves credential/foothold references and revalidates the command. Local mode uses argument arrays with `create_subprocess_exec`; remote mode preserves literal argument semantics through quoting and requires a trusted SSH fingerprint.

Heartbeats maintain job ownership. Browser disconnection leaves a durable job running; cancellation is a separate action. A stale lease marks work interrupted. A late result must still match its owner and attempt before it can publish. Terminal attempts are archived before another approval retries the step.

```
operator approves eligible step
claim assessment and enqueue execution attempt
worker conditionally claims queued job
resolve references; recheck scope, policy and phase
execute with time/output bounds; heartbeat ownership
publish only while ownership and attempt still match
encrypt originals, store redacted projection and archive result
```

Timeout, interruption and cancellation are explicit outcomes. ZAP baseline exit codes 1 and 2 indicate completed scans with findings; other nonzero exits normally indicate failure. Failure is not evidence that a target is clean. The shared SSH tmux lock protects one process only.

5. Protected evidence and streaming

Sources: `evidence.py`, `secret_store.py`, `jobs.py`.

Original command, stdout and stderr are encrypted together; archived attempts retain their own originals. Provenance hashes the original output pair. Internal verification and campaign ingestion can read originals. Ordinary API responses, provider analysis, findings and report evidence receive redacted text.

Redaction recognizes supported credential tables, labeled secrets, private keys, URL credentials and known recovered values. It is not a classifier for all unlabelled strings. Plans use `$$loot:<id>$$` for known recovered credentials;

operator-authored plan payloads remain executable text.

The private live buffer keeps the beginning of the stream up to its bound, retaining table context. Only complete redacted lines are published. Partial or overlong lines are withheld; this is not a sliding raw-output tail.

```
capture bounded originals privately
compute hash from original stdout/stderr
encrypt original command and streams
derive public redacted fields
for reveal:
    authenticate and decrypt strictly
    commit evidence_access audit
    return original with Cache-Control: no-store
```

Invalid operator keys return 401. Missing/wrong encryption keys or unreadable originals produce a conflict. The system must not replace protected originals with empty values after decryption failure.

6. Analyzer normalization and deduplication

Source: `analyzer.py`.

Streams are bounded, stripped of ANSI/control artifacts and dispatched to deterministic parsers for Nmap, Nuclei, curl, WhatWeb, SSLScan, sqlmap, Searchsploit, Metasploit and passive ZAP. Provider enrichment uses redacted, bounded input and cannot override authorization.

Normalization constrains severity, numeric drivers, location strings and text length. Fingerprints combine normalized title, endpoint and parameter. Duplicates merge their strongest bounded drivers/severity and source-tool provenance before rescoring. Nmap endpoints retain host identity so hosts in a sweep are not accidentally merged.

```
consume each stdout then stderr within shared input budget
run deterministic tool parsers
optionally merge validated provider findings
normalize fields and bound result count
group by finding fingerprint
merge duplicate evidence, drivers and source tools
compute risk and priority from normalized values
```

7. Independent verification

Source: `verification.py`.

The confirming adapter recognizes explicit successful sqlmap injection proof. It derives the primary endpoint from the executed command, requires compatible vulnerability category and parameter, and compares scheme, hostname, effective port, case-sensitive path and remaining query parameters.

Failed runs, tentative or negated claims, ambiguous endpoints and secondary-URL runs cannot establish confirmation. HTTP 200, service banners, exploit catalog matches and provider-authored text do not establish it either. Replacing exploitation analysis clears dependent verification before relinking.

8. Atomic analysis and phase advancement

Sources: `main.py`, `workflow.py`, `footholds.py`.

Every enabled current-phase step needs a terminal result. Analysis captures the workflow revision and execution attempts, computes from that snapshot, then claims the matching revision before publication. Helpers flush; the caller owns the transaction.

```
capture eligible phase, revision and terminal attempts
compute normalized findings from bounded evidence
revalidate snapshot and claim expected workflow revision
replace current-phase findings and verification links
ingest campaign evidence from protected originals
advance phase and draft eligible next work
commit all state together; on exception roll back
```

A slow computation cannot overwrite a newer plan or execution result. Conflict does not cause an automatic scanner retry. Prerequisite and permission refusals can be returned as drafting notes instead of inventing executable next steps.

9. Campaign ingestion and promotion

Sources: `footholds.py`, `planners.py`, `scope_rules.py`, `main.py`.

Recognized output creates bounded host, foothold and encrypted loot records. Credential fingerprints support deduplication; list responses mask secrets. References resolve immediately before execution without rewriting the approved plan into a plaintext secret store.

Promotion requires an owned host in the same campaign, parent host-wide authorization and no exclusion overlap. The child inherits restrictions and a scope limited to the host. URL/path/port-only grants cannot justify the child's whole-host recon plan. A conditional revision claim prevents two concurrent requests from promoting the same host.

10. Recommendations, correlations and autopilot

Sources: `next_steps.py`, `attack_paths.py`, `auto_recommend.py`.

A bounded engagement digest summarizes findings, executions and campaign records. Candidates are limited, checked for known evidence provenance, deduplicated against existing commands and policy-validated. Different flags for the same tool/host can represent legitimate deepening.

Attack-path candidates group related findings by location/parameter and require meaningful shared evidence. Rank is based on summed member priority. The result is a correlation, not a probability or proof that a chain succeeded. Provider narratives must cite real member findings; invalid provenance falls back to deterministic text.

Opt-in autopilot ingests/drafts and records local progress atomically. It stops at its iteration bound or no-progress condition. Advisory provider work occurs after commit and is saved only if the committed revision remains current. Approval, execution and operator analysis remain explicit actions.

11. Immutable report publication

Sources: `reporter.py`, `report_versions.py`, `workflow.py`.

Reports include bounded redacted evidence and provenance. Slow rendering writes a unique staging file. Publication checks the captured revision and creates an immutable digest-named file plus a database report version/current pointer.

```
snapshot assessment revision, evidence and provenance
render HTML to a unique staging file
claim unchanged revision
publish immutable report bytes and version row
commit current_report_version_id
clean failed staging; retire obsolete public paths after commit
```

A stale report returns conflict and cannot reset newer state to reported. Historical public versions retain their digests. Pre-upgrade reports are restricted: their originals remain encrypted behind audited reveal, while a newly generated version supplies the redacted public download.

12. Startup, deletion and validation

Startup runs Alembic upgrades, settings migration and evidence backfill. External workers wait for both schema head and complete protected-evidence adoption. Keep API, worker, database and encryption key compatible.

Campaign deletion follows foreign-key dependencies. A parent assessment with children returns conflict; remove children first. Workspace reset removes the graph while retaining settings and refuses active work.

The testing guide records isolated fault/concurrency regression results and their limits. The deployment guide covers key sharing, readiness and live scanner/VM checks. These algorithms do not replace deployment validation.